Informe Técnico: Módulo de Negociación con Spring Boot

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Esta evidencia codifica un módulo del proyecto **MONK GRAIN** aplicando un framework Java. El módulo elegido es el **mercado en su segundo sentido**: el comprador publica lo que busca y los caficultores responden con su precio.

Es la diferencia entre esperar a que aparezca el café que uno quiere y pedirlo. En el catálogo —evidencias AA2-EV01 y AA2-EV02— el vendedor publica lo que tiene; aquí ocurre al revés, y por eso completa el producto en lugar de repetirlo.

2. Objetivo

Codificar un módulo del proyecto aplicando uno de los frameworks vistos, usando los artefactos previos —historias de usuario, modelo de datos y diagrama de clases—, incluyendo comentarios, cumpliendo los estándares de codificación y trabajando con Git.

3. Por qué Spring Boot

La guía pide «uno de los frameworks vistos». Se escogió Spring Boot porque resuelve de una vez cuatro cosas que en el módulo anterior hubo que escribir a mano:

Qué aporta	Cómo se hacía en AA2-EV01
Inyección de dependencias	Cada clase construía sus colaboradores con `new`
Acceso a datos con JPA	Cada consulta era una cadena SQL y un `PreparedStatement`
Validación declarativa	Un método `validate()` con una cadena de condicionales
Transacciones	Manejo explícito de `commit` y `rollback`

> **Lo que el framework también esconde.** Spring Data genera las consultas a partir del nombre de los métodos, lo que es cómodo hasta que falla. Por eso las consultas con reglas de negocio se escriben explícitas con `@Query`, en lugar de confiarlas a un nombre de método de cuarenta caracteres que nadie sabe leer.

4. El dominio: el mercado de dos sentidos

Una solicitud de compra pasa por cinco estados, y uno de ellos existe por una razón del negocio que conviene entender:

Estado	Qué significa
`OPEN`	Admite posturas y no se ha cubierto nada
`PARTIALLY_COVERED`	Se aceptó parte de lo pedido y falta el resto
`FULFILLED`	Se cubrieron todos los kilos
`CLOSED`	El comprador cerró con algo cubierto
`CANCELLED`	El comprador la retiró sin aceptar nada

`PARTIALLY_COVERED` no es simetría, es el negocio. En pergamino lo normal es que una solicitud grande no la cubra un solo caficultor: quien busca diez cargas suele armarlas con tres vendedores. Marcarla como cumplida al aceptar la primera postura cerraría la solicitud cuando todavía falta café.

Y cerrar sin haber cubierto nada **es cancelar**, no cerrar. La distinción importa en el historial del comprador: una cosa es que consiguiera su café en varias partes y otra que se arrepintiera.

5. Arquitectura

```
controller  →  service  →  repository  →  JPA  →  PostgreSQL
    │            │
  valida       reglas del
 la entrada     negocio
```

Paquete	Responsabilidad
`model`	Entidades JPA con sus lecturas derivadas
`dto`	Lo que el cliente puede enviar — ver §7
`repository`	Interfaces de Spring Data; sin implementación propia
`service`	Las reglas del mercado y las transacciones
`controller`	La API y la traducción de errores a HTTP

5.1 Las dependencias llegan por el constructor

```
public NegotiationService(PurchaseDemandRepository demandRepository,
                          SellerOfferRepository offerRepository) {
    this.demandRepository = demandRepository;
    this.offerRepository = offerRepository;
}
```

Y no por campo con `@Autowired`. Así el objeto no puede existir a medio construir, y en las pruebas los colaboradores se sustituyen sin necesidad del contenedor de Spring.

6. Validación declarativa

La regla vive junto al dato que restringe, no en otro archivo que hay que recordar actualizar:

```
@Min(value = 1, message = "Los kilos que buscas deben ser al menos 1.")
@Column(name = "wanted_kg", nullable = false)
private int wantedKg;

@DecimalMin(value = "80.0", message = "El factor de rendimiento debe estar entre 80 y 130.")
@Column(name = "max_yield_factor", precision = 5, scale = 2)
private BigDecimal maxYieldFactor;
```

Los mensajes están en español porque los lee una persona en Colombia, y nombran el campo y el rango: un «must not be blank» no le dice a nadie qué corregir.

7. Dos defectos encontrados al ejercer la API

Ninguno aparece al leer el código ni al pasar las pruebas: los dos salieron al enviar peticiones reales.

7.1 La entidad como cuerpo de la petición permitía sobrescribir otra postura

El controlador recibía directamente `SellerOffer`. Eso deja que el cliente escriba cualquier campo, incluidos `id` y `status`. Un `{"id": 5}` convertía el alta en una **sobrescritura de la postura 5**, porque `save()` actualiza cuando el identificador ya existe.

La corrección son los objetos de entrada `DemandRequest` y `OfferRequest`: lo que no está declarado ahí no se puede enviar.

Comprobado — se envía id y status ajenos: POST {"id":5,"status":"ACCEPTED","sellerName":"Atacante", …} → se creó id=6 con status=PENDING → la postura 5 siguió intacta y en PENDING

7.2 Un `@NotNull` sobre un campo que pone el servidor

`SellerOffer.demand` llevaba `@NotNull`. Ese campo no lo envía el vendedor —lo establece el servicio a partir de la ruta—, de modo que la validación corría antes de la asignación y **rechazaba toda postura válida** con un «must not be null» que no decía nada:

```
422 → {"codigo":"validacion","campos":{"demand":"must not be null"}}
```

Se retiró la anotación. La integridad la garantiza `nullable = false` en la columna, que es donde corresponde.

8. Tres trampas del framework, con su motivo

Las tres costaron tiempo y las tres son del tipo que vuelve si no queda escrito por qué está así.

8.1 Un getter derivado rompió una consulta de Spring Data

Al añadir `getDemandId()` a la entidad, el repositorio dejó de arrancar:

```
Could not resolve attribute 'demandId' of 'SellerOffer'
```

Spring Data resolvía `existsByDemandIdAndSellerName…` como el recorrido `demand.id`. Con el getter presente pasó a buscar un atributo llamado «demandId», que existe como método pero no como columna. La corrección es el guion bajo explícito — `existsByDemand_IdAndSellerName…` —, que le dice que recorra la relación, más `@Transient` sobre el getter.

8.2 La colección perezosa y el ciclo de serialización

`PurchaseDemand.offers` es perezosa y `open-in-view` está desactivado —que es lo correcto—, así que cargarla al escribir la respuesta fallaba con «could not initialize proxy». Y aunque cargara, cada postura apunta de vuelta a su solicitud: Jackson entraría en un ciclo infinito.

Se marca con `@JsonIgnore` en ambos sentidos, y la postura expone `demandId` en lugar de la solicitud entera.

8.3 `@Validated` hacía que la validación fallara por el camino equivocado

Rechazar una postura sin motivo devolvía un 400 con este cuerpo:

```
"Validation failed for method='public …rechazar(java.lang.Long,java.lang.String)'.
 Error count: 0"
```

Delata la firma del método y no dice qué corregir. Al añadir un manejador para `HandlerMethodValidationException` y `@Validated` a la vez, el resultado empeoró a **500**: `@Validated` activa el interceptor AOP heredado, que lanza `ConstraintViolationException`, mientras que desde Spring Framework 6.1 el controlador **ya valida por sí solo** y lanza la otra. Con los dos activos, la validación ocurre dos veces y el error llega por el camino que el manejador no espera.

La corrección: quitar `@Validated` y quedarse con el mecanismo nativo. Hay además un detalle de versión que conviene anotar: el método del resultado es `getAllValidationResults()` en Spring 6.1 y `getParameterValidationResults()` desde 6.2.

9. Verificación en ejecución

Desplegado sobre PostgreSQL y ejercido por HTTP real.

```
=== 1. El comprador publica lo que busca ===
  solicitud 4: Pergamino factor 92 o mejor, 10 cargas
  busca 1250 kg (10 cargas) · OPEN · faltan 1250 kg

=== 2. Tres caficultores posturan ===
  postura 7: Finca El Mirador          750 kg a 24500.0 · Excepcional
  postura 8: Cooperativa de Narino     500 kg a 25200.0 · De exportación
  postura 9: Hacienda Alta Vista       250 kg a 23800.0 · De exportación

=== 3. El comprador las ve, la más barata arriba ===
  [PENDING ] Hacienda Alta Vista       250 kg a 23800.0
  [PENDING ] Finca El Mirador          750 kg a 24500.0
  [PENDING ] Cooperativa de Narino     500 kg a 25200.0

=== 4. Acepta dos: cubre 1000 de 1250 ===
  PARTIALLY_COVERED · cubierto 1000/1250 kg (80 %) · faltan 250

=== 5. La de 500 ya no cabe ===
  422 → Ya solo faltan 250 kg: se aceptaron otras posturas mientras tanto.

=== 6. Una cuarta cubre los 250 que faltan ===
  FULFILLED · cubierto 1250/1250 kg (100 %)

=== 7. La pendiente se rechaza sola, con motivo ===
  [ACCEPTED] Hacienda Alta Vista
  [ACCEPTED] Finca El Mirador
  [ACCEPTED] Finca La Esperanza
  [REJECTED] Cooperativa de Narino  La solicitud se cubrió con otras posturas.
```

El paso 7 es el que más importa: cuando la solicitud se cubre, las posturas que seguían esperando **se rechazan con su motivo**. Dejarlas pendientes para siempre es peor que un «no», porque el vendedor sigue contando con esos kilos.

9.1 Validaciones por HTTP

Entrada	Código	Respuesta
Posturar sobre solicitud cumplida	`422`	Esta solicitud ya no admite posturas.
Factor 145	`422`	`yieldFactor`: El factor de rendimiento debe estar entre 80 y 130.
Precio en cero	`422`	`pricePerKg`: El precio por kilogramo debe ser mayor que cero.
Sin vendedor ni lote	`422`	Los **dos** campos a la vez, no el primero
Solicitud inexistente	`404`	No existe la solicitud 999.
Rechazar con motivo vacío	`400`	`motivo`: Indica por qué rechazas la postura.
Rechazar sin el parámetro	`400`	Falta el parámetro «motivo».
Responder una ya respondida	`422`	Esta postura ya fue respondida: está en estado ACCEPTED.

Los códigos no son intercambiables: `404` cuando no existe, `400` cuando el parámetro está mal escrito, `422` cuando el dato está bien escrito pero la situación no lo permite.

10. Pruebas

21 pruebas con JUnit 5 y Spring Boot Test, contra una base H2 en memoria: comprobar una regla de negocio no debe exigir un PostgreSQL vivo.

Grupo	Qué fija
Publicar (3)	Que nace abierta, que **no se pueda declarar cubierta** desde fuera, y la lectura en cargas
Posturar (5)	Ofrecer más de lo que falta, pasarse del precio máximo, repetir lote, y solicitud vencida
Responder (5)	Cubrir en partes, que al cubrirse se rechacen las pendientes, rechazo sin motivo, y responder dos veces
Retirar y cerrar (5)	Que nadie retire la postura de otro, y que cerrar sin cubrir sea cancelar
Compatibilidad (3)	Que un factor mejor entre, uno peor no, y uno sin declarar lo decida el comprador

> **Lo que puedes hacer tú**
>
> **Las capturas y el video de la evidencia AA3-EV02**, que pide un pantallazo por cada requisito. El módulo queda corriendo en **http://localhost:8081/api/negociacion/solicitudes**; se levanta de nuevo con `java -jar target/negociacion-pergamino.jar`. Las peticiones de la sección 9 sirven de guion.

11. Conclusión

El módulo implementa el mercado en su segundo sentido aplicando Spring Boot: inyección por constructor, acceso a datos con Spring Data JPA, validación declarativa sobre el modelo, transacciones por anotación y traducción de errores a HTTP en un solo sitio.

Lo que más enseñó no fue lo que el framework resuelve, sino lo que esconde: un getter derivado que rompe una consulta generada por el nombre del método, una colección perezosa que falla al serializar, y dos mecanismos de validación que al activarse a la vez convierten un 400 en un 500. Las tres quedan escritas en el código, porque son exactamente el tipo de decisión que alguien deshace si no sabe por qué está ahí.